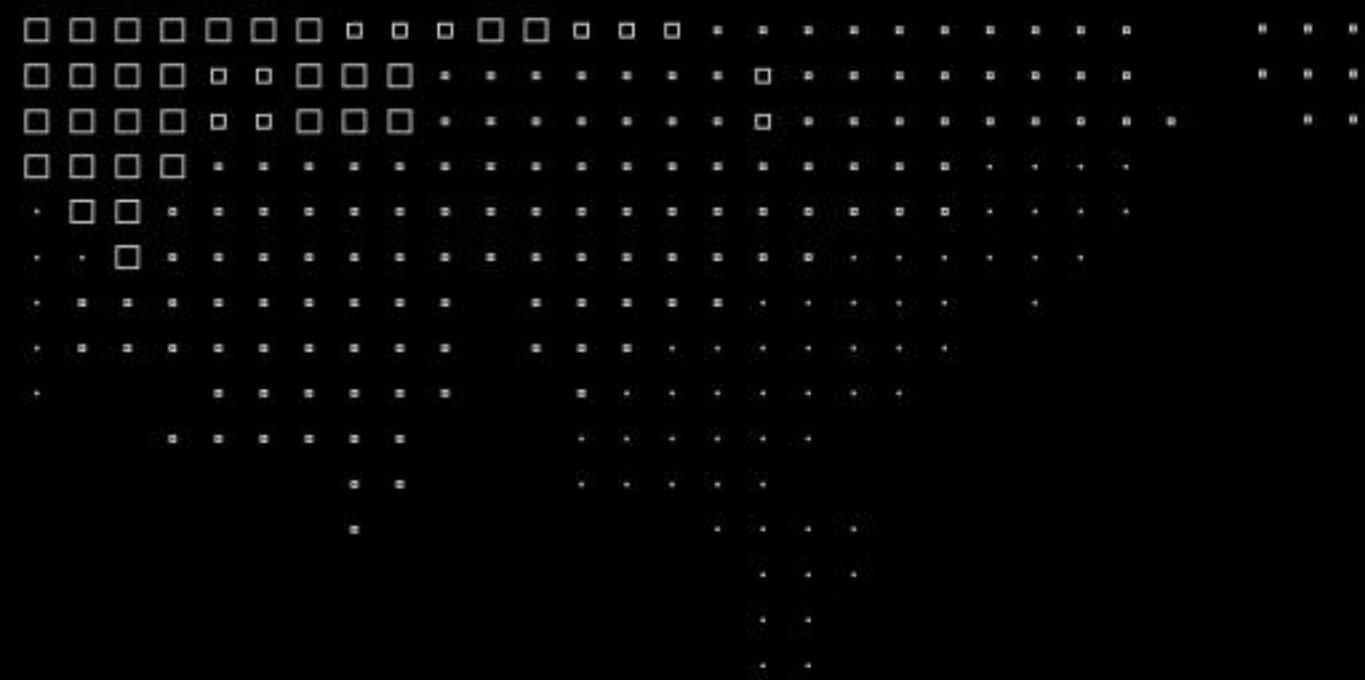
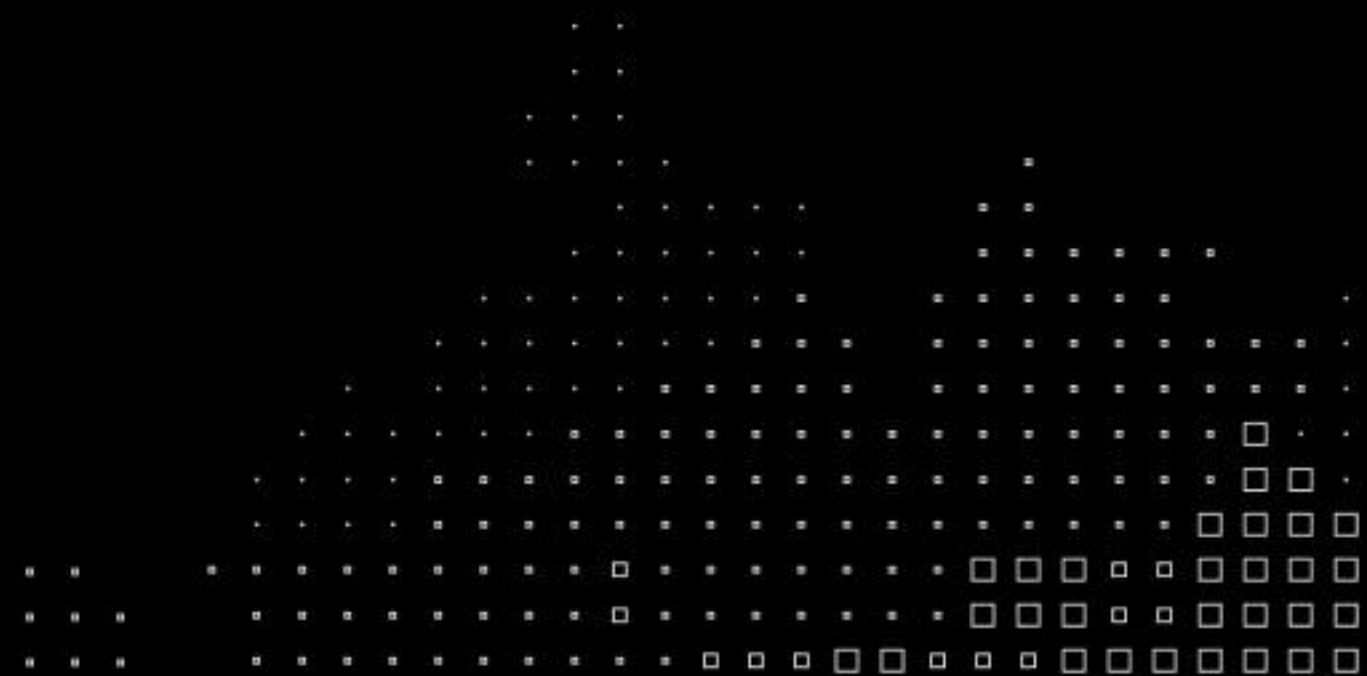




Ciência da Computação



DSA

Data Structures and Algorithms

Professor Álvaro Gonçalves
Email: profalvaro.goncalves@fiap.com.br

Conteúdo do Semestre 1



Data	Encontro	Conteúdo
02-06/03	E1	Apresentação + Problema → Dado → Estrutura + ADT
09-13/03	E2	Big-O intuitivo (custo por operação)
16-20/03	E3	Arrays — acesso, inserção, custo
23-27/03	E4	Listas encadeadas — conceito e pseudocódigo
30-03/04	E5	Stack / Queue (ADTs lineares)
06-10/04	REV1	Revisão para CP1
13-17/04	CP1	Checkpoint 1 — ADTs lineares
20-24/04	E6	Recursão (modelo)
27-30/04	E7	Busca: Linear × Binária
04-08/05	E8	Ordenação $O(n^2)$
11-16/05	REV2	Revisão para CP2
18-22/05	CP2	Busca + Ordenação + Big-O
25-29/05	E10 (AVF)	Desafio final do semestre

Revisão de Fundamentos: O Caminho para Vetores

ALGORITMOS (Receitas)

Sequência de passos finita e bem definida para resolver um problema.

Exemplo Pseudocódigo (Média):

```
Início
  Leia num1, num2
  soma = num1 + num2
  media = soma / 2
  Escreva media
Fim
(Impl. JS em aula)
```

ADTs (Tipos Abstratos de Dados)

Modelo que define ***o que*** um dado faz, não ***como*** (ex: **Lista, Pilha**).

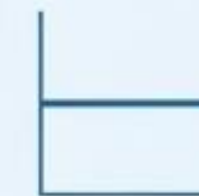
Exemplo Pseudocódigo (ADT Lista):

```
Operações:
  adicionar(item)
  remover(item)
  obter(índice)
  tamanho()
(Comportamento Abstrato)
```

NOTAÇÃO BIG-O (Eficiência)

Mede como o tempo/espaco cresce com a entrada (N). Foco no pior caso.

Exemplos Big-O:



O(1)
Constante
(ex: acesso
vetor)



O(N)
Linear
(ex: busca
simples)

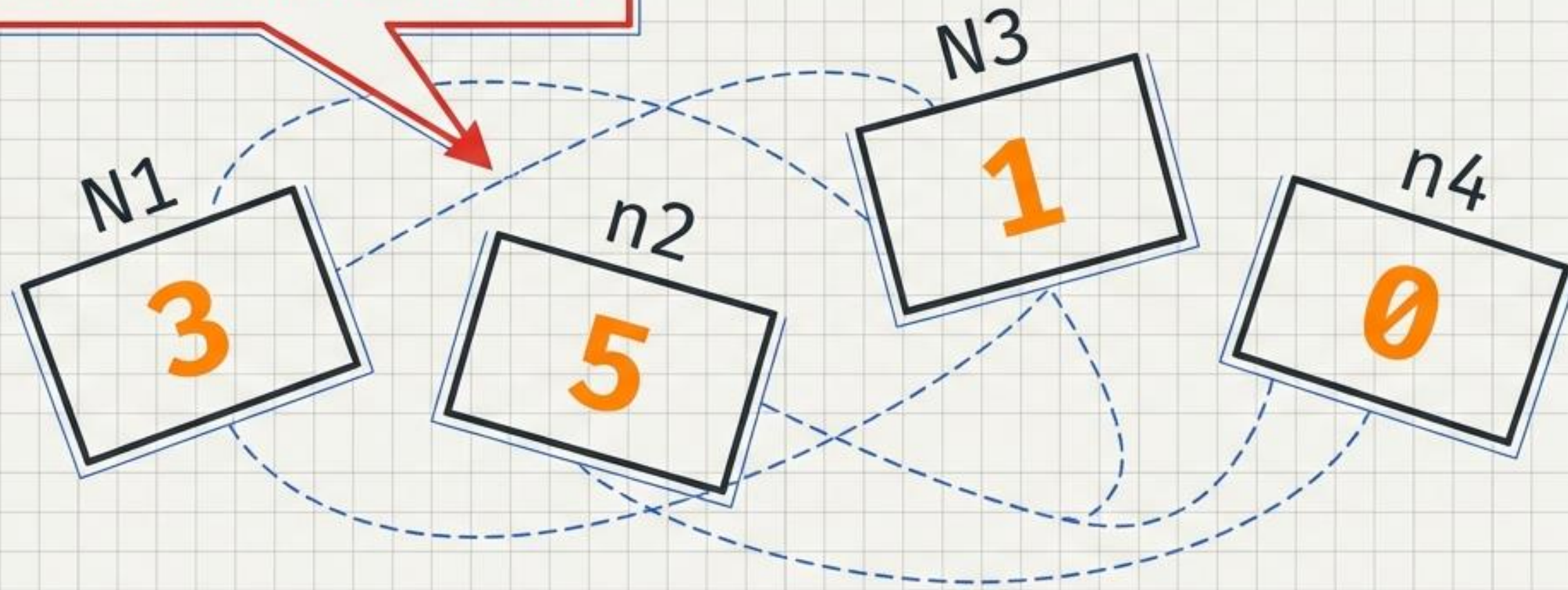


O(N²)
Quadrático
(ex: loops
aninhados)

→ Introdução a VETORES: Estrutura eficiente (O(1) acesso) que implementa ADT Lista.

O Paradigma das Variáveis Simples

Alocação Aleatória. O sistema operacional reserva espaços espalhados e desconectados no 'armário' da memória.



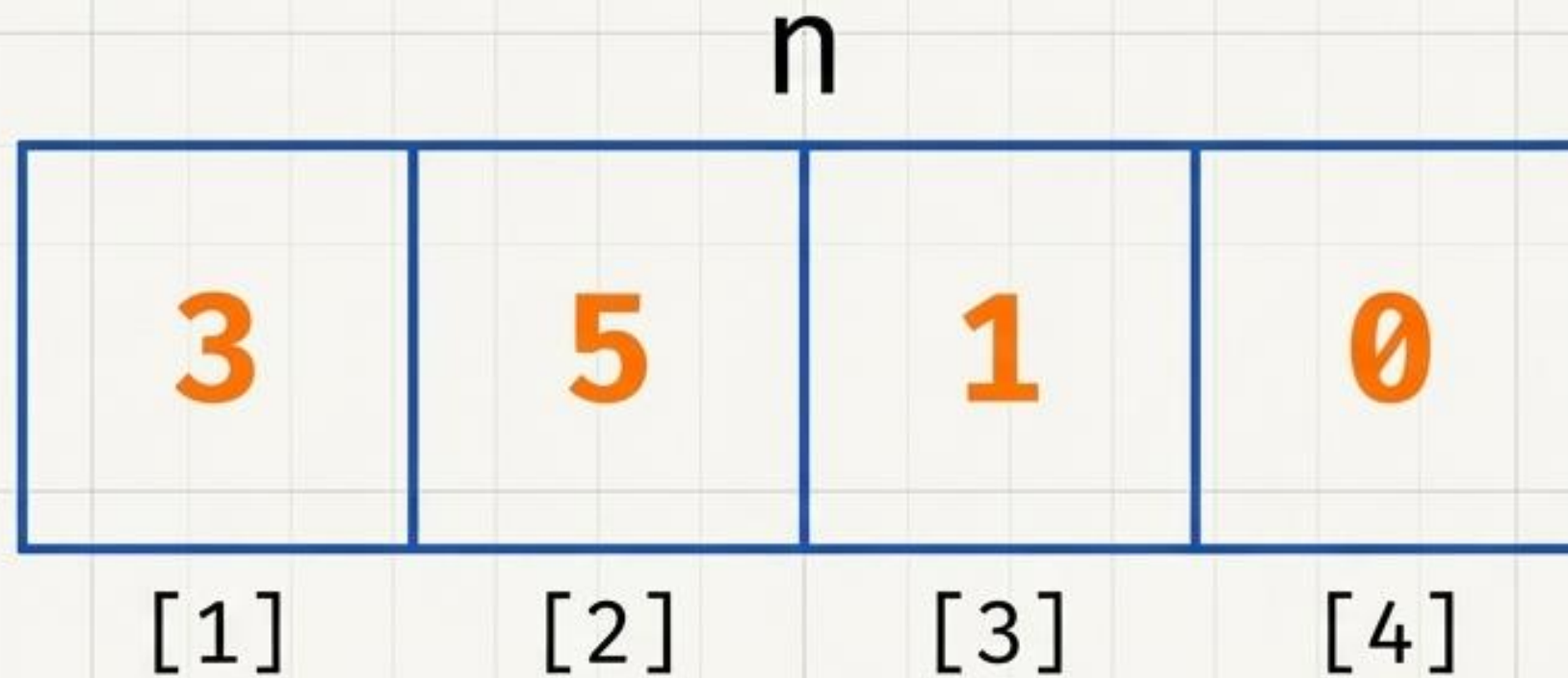
Para armazenar 4 valores, precisamos de 4 declarações independentes.
Para 1000 valores? Um pesadelo logístico.

O Preço da Simplicidade: Amnésia Digital



Sem a capacidade de armazenar múltiplos estados sob o mesmo contexto, algoritmos não conseguem olhar para trás para analisar padrões (como encontrar posições de números pares).

A Evolução Estrutural: O Vetor



`n: vetor [1..4] de inteiro`

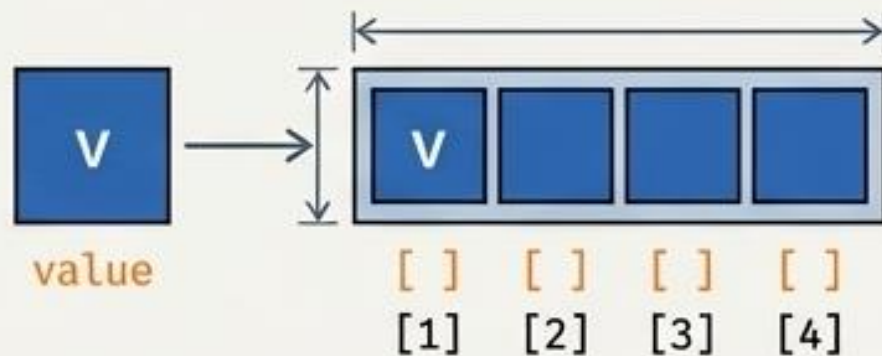
Em vez de 4 variáveis para 4 valores, criamos 1 variável com 4 espaços contíguos. A memória agora é um sistema organizado e mapeado.

Decodificando a Nomenclatura

Variável Composta Homogênea Unidimensional

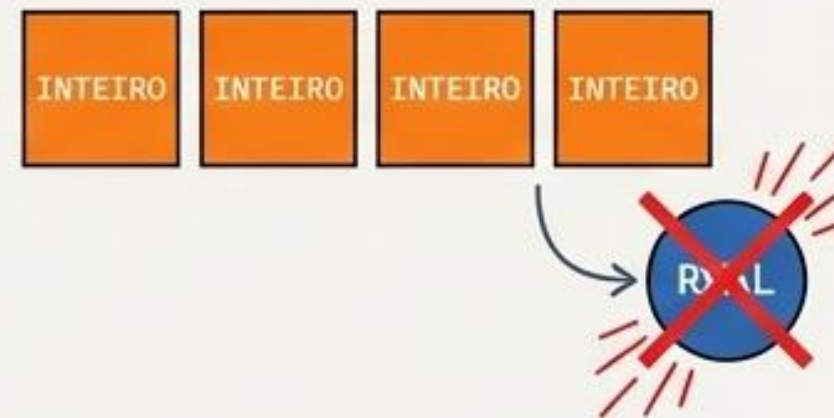
Múltiplos Espaços

Possui várias casas sob um único nome.



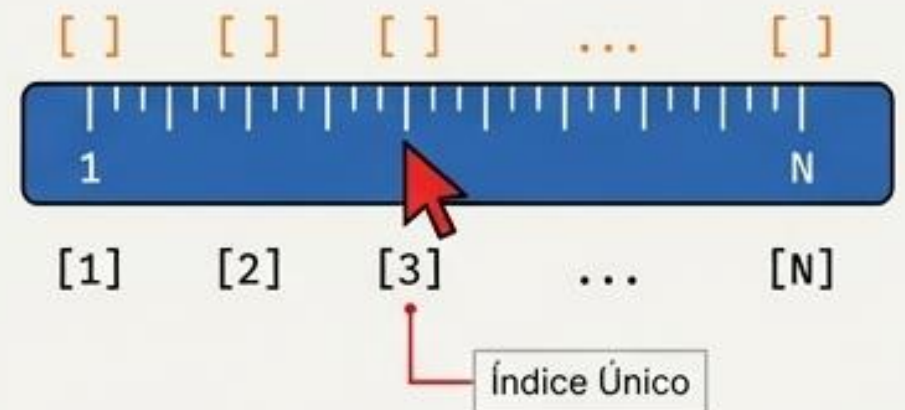
Tipagem Única

Aceita apenas um formato de dado. Se é Inteiro, todos os espaços devem ser Inteiros.

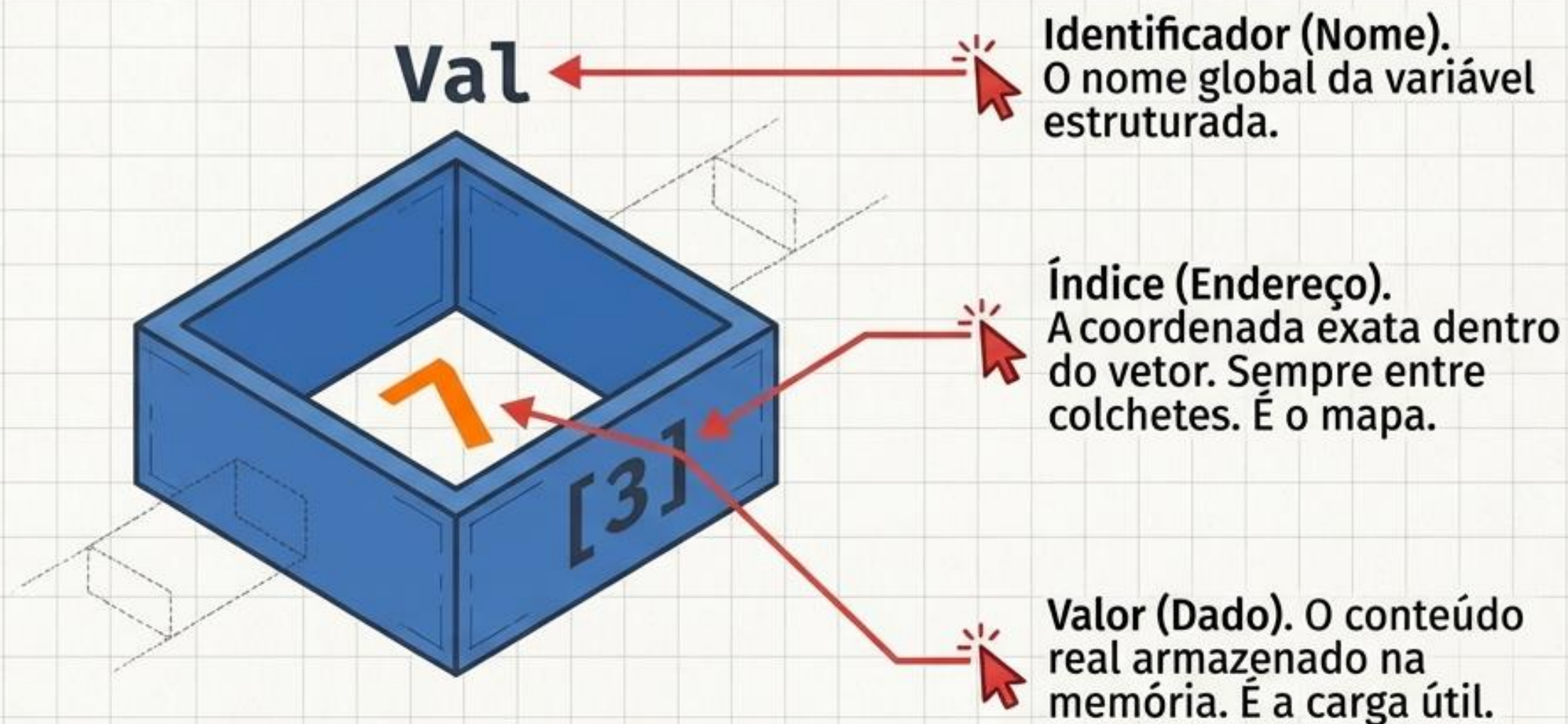


Linha Reta

Exige apenas um endereço (índice) para localizar o dado, indo da posição 1 a N.



Anatomia de uma Posição



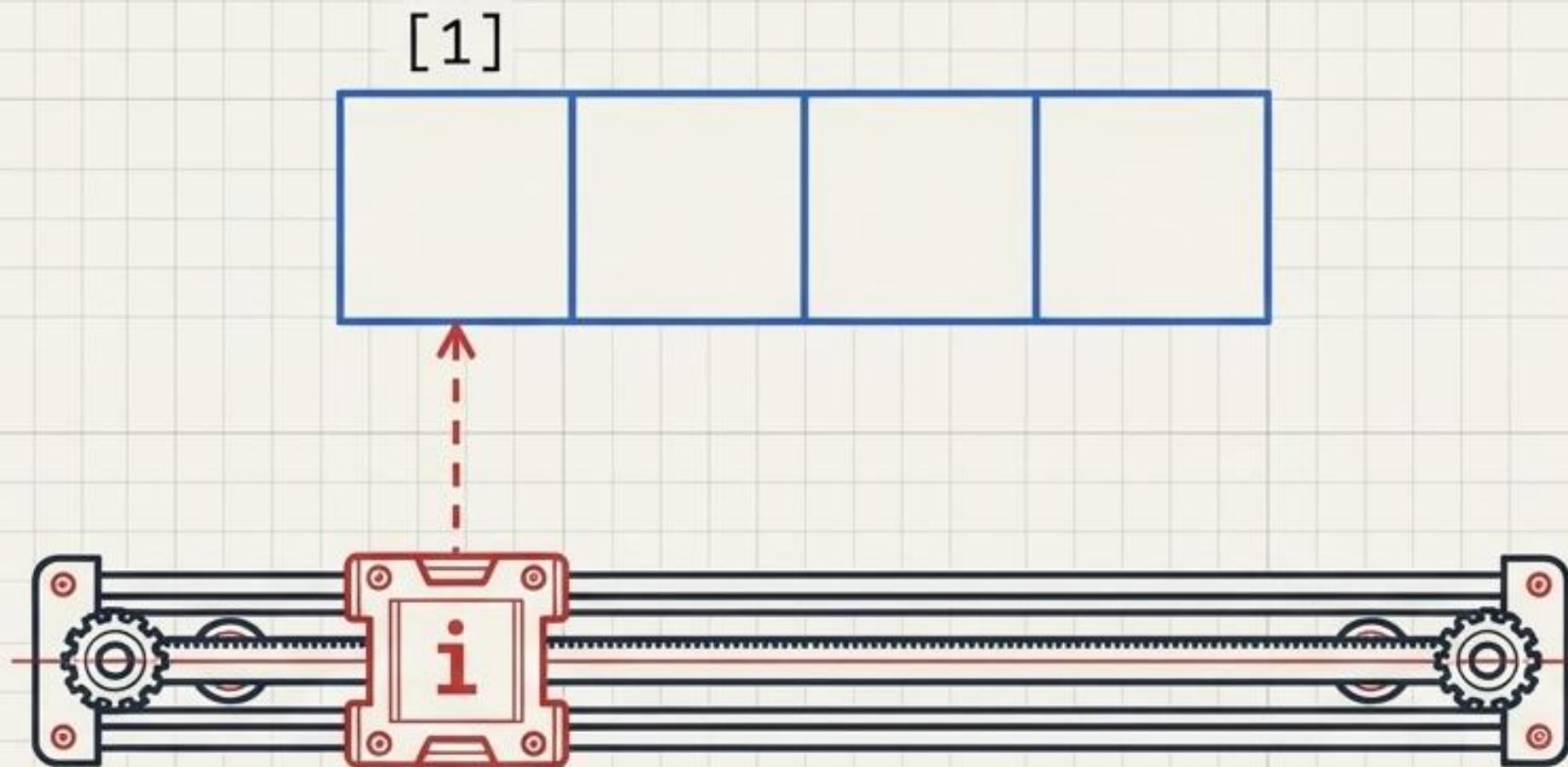
`Val[3] ← 7`

(Lê-se: O vetor Val na posição 3 recebe o valor 7).

O Ponto de Virada

	Variáveis Simples 	Vetores 
Alocação	Bagunçada e Dispersa	Contígua e Estruturada
Declaração	1 linha por dado (Ex: N1, N2...) Fira Code	1 linha para N dados (Ex: Vetor [1..100]) Fira Code
Acesso em Lote	Manual, um a um	Automatizado via Laço de Repetição
Histórico	Sobrescrito (Perdido)	Preservado na Memória

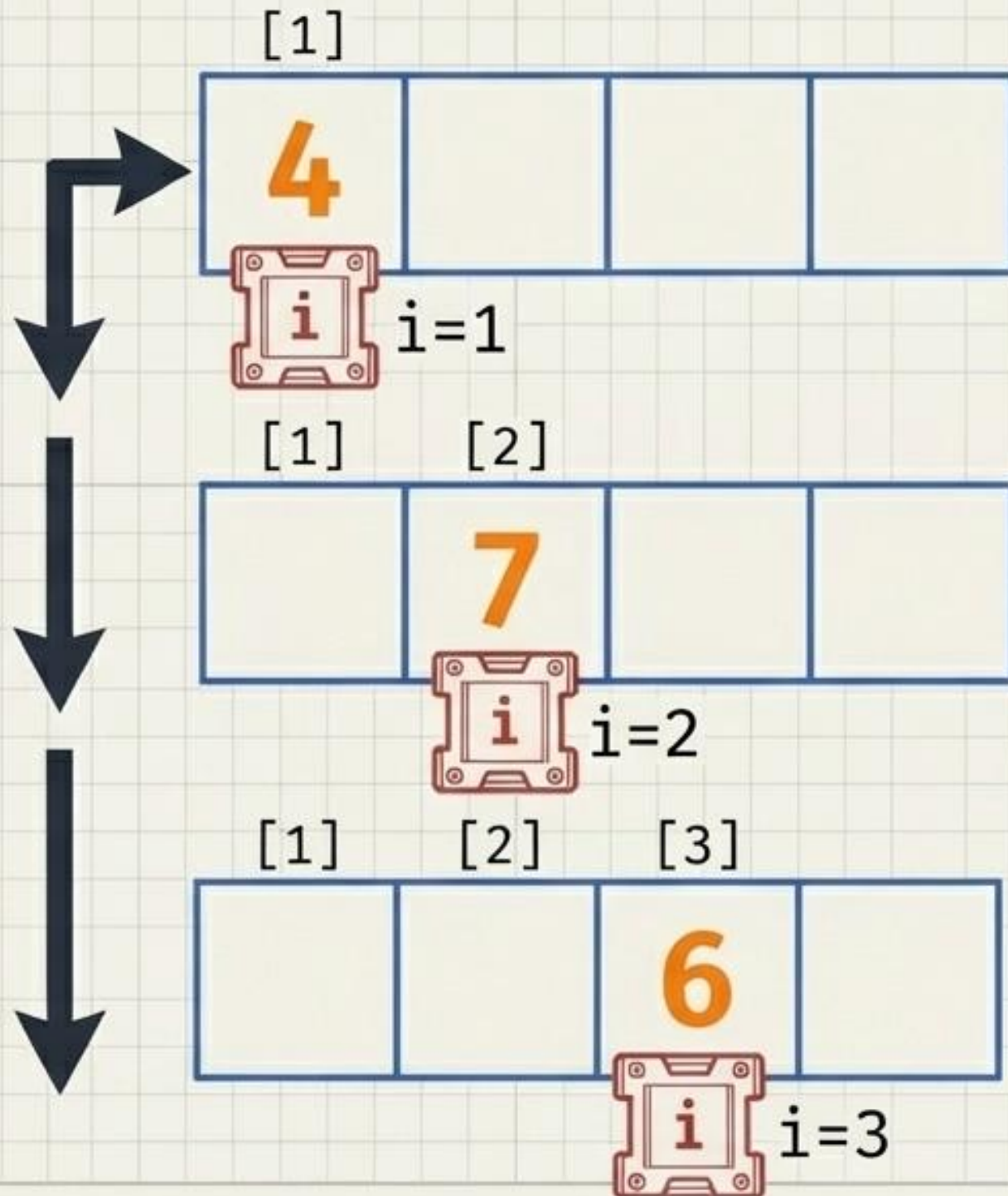
O Cursor Dinâmico: A Variável "i"



Para interagir com um vetor, precisamos de um "apontador". A variável simples `i` atua como um cursor que desliza pelas posições do vetor, da casa 1 até a última.

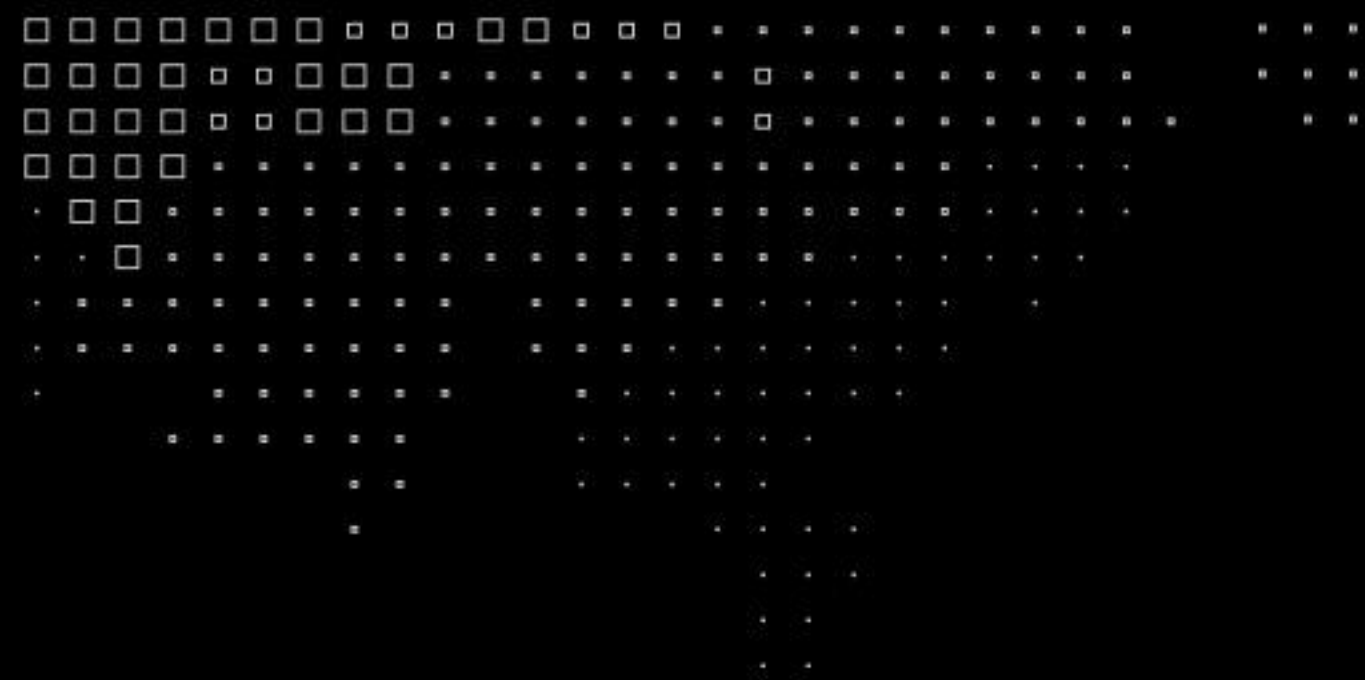
Quando `i = 1`, `Val[i]` é exatamente o mesmo que `Val[1]`.

Automatizando a Entrada de Dados



```
para i de 1 ate 6 faca  
    leia(v[i])  
fimpara
```

O laço para (for) incrementa o índice automaticamente. Cada ciclo do laço move o cursor uma posição para a direita, preenchendo o vetor sem precisarmos escrever múltiplos comandos leia separados.



Copyright © **2026** Prof. Álvaro Gonçalves

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor Álvaro Gonçalves.

